

GeoPulse: *A Real-Time Satellite Monitoring System*

GeoPulse is an open-source platform designed to automate the **ingestion, processing, and analysis** of satellite imagery (Landsat, Sentinel, MODIS, etc.). It provides **real-time insights** into environmental changes, including vegetation health, surface temperature variations, and urban expansion.

The system integrates modern data pipelines, cloud processing, and GIS visualization to empower researchers, policymakers, and developers with accessible geospatial intelligence.

The system is designed with modular subsystems.

Subsystems

1. Data Ingestion Subsystem

 **Purpose:** Acquire and organize satellite imagery for user-defined AOIs and timeframes.

Components:

-  **AOI Manager:** Handles user-defined Area of Interest (AOI) through uploaded GeoJSON/shapefile or drawn polygons.
-  **Scheduler:** Orchestrates periodic imagery fetch (weekly/monthly) using Airflow or Prefect.
-  **Event Listener:** Detects new satellite datasets from USGS or Google Earth Engine APIs.
-  **Ingestion Queue:** Kafka or RabbitMQ topic (**raw_imagery**) to stream incoming data requests.
-  **Storage Layer:** Stores raw imagery in S3/local blob store; metadata in PostgreSQL/PostGIS.

Interfaces:

- **Input:** AOI definitions + API requests (USGS/EE)
 - **Output:** **raw_imagery** message queue + metadata database
-

2. Preprocessing Subsystem

 **Purpose:** Clean, normalize, and prepare imagery for analysis.

Components:

-  **Worker Pool:** Asynchronous Celery/Kafka consumers that process imagery tasks.
-  **Operations:** Cloud masking, band extraction, atmospheric correction, AOI clipping.
-  **Output Format:** GeoTIFF, NetCDF, or COG (Cloud-Optimized GeoTIFF).

- ⚡ **Caching Layer:** Redis for storing intermediate NumPy arrays or temporary results.

🔗 Interfaces:

- **Input:** `raw_imagery` queue
 - **Output:** `clean_imagery` queue
-

✅ 3. Index Computation Subsystem

🎯 **Purpose:** Compute spectral and thermal indices (NDVI, NDWI, NDBI, LST).

🧩 Components:

- 📊 **Index Processors:** Independent workers that calculate environmental indices using Rasterio + NumPy.
- 📈 **Algorithms:** NDVI, NDWI, NDBI, LST (configurable formulas).
- 🧠 **Parallel Execution:** Distributed execution via Dask or Spark.
- 🕒 **Version Control:** Each computation is tagged with AOI ID, timestamp, and index version.

🔗 Interfaces:

- **Input:** `clean_imagery` queue
 - **Output:** `index_results` topic
-

✅ 4. Change Detection & Anomaly Subsystem

🎯 **Purpose:** Identify significant environmental or land-use changes.

🧩 Components:

- 🔍 **Comparators:** Analyze multi-temporal imagery to detect deviations.
- 🚦 **Rules Engine:**
 - NDVI drop > 20% (vegetation loss)
 - LST spike > 5°C (temperature anomaly)
 - NDBI increase (urban expansion)

- 📢 **Output:** JSON alerts with AOI ID, timestamp, and severity.

🔗 Interfaces:

- **Input:** `index_results` topic
 - **Output:** `alerts` topic
-

✅ 5. Notification & Alerting Subsystem

🎯 **Purpose:** Deliver alerts to users when anomalies occur.

🧩 Components:

- 📧 **Notification Service:** Consumes `alerts` topic and formats alert messages.
- 🔔 **Channels:** Email (SMTP), Slack, or REST Webhooks.
- 📄 **Templates:** Configurable severity levels and message formats.

🔗 Interfaces:

- **Input:** `alerts` topic
 - **Output:** User-facing endpoints (email, Slack, API).
-

✅ 6. Storage & Caching Subsystem

🎯 **Purpose:** Maintain structured, time-series, and cached data for fast access.

🧩 Components:

- 🗄️ **Time-Series DB:** TimescaleDB or InfluxDB for long-term trend storage.
- ⚡ **Cache Layer:** Redis for frequently accessed index data.
- 📦 **Archival Storage:** S3 or local volume for historical imagery and analysis results.

Interfaces:

- **Input:** `index_results`, `alerts`
 - **Output:** APIs + visualization dashboards
-

7. Visualization Subsystem

 **Purpose:** Display geospatial analytics, temporal trends, and predictions.

Components:

-  **Frontend:** Streamlit or React + Leaflet for interactive map dashboards.
-  **Backend:** FastAPI serving REST & WebSocket endpoints.
-  **Live Updates:** Server-Sent Events (SSE) or WebSocket streams for real-time updates.
-  **Map Layers:** Dynamic overlays for NDVI, LST, and urban expansion maps.

Interfaces:

- **Input:** Database, Redis, or WebSocket feeds
 - **Output:** User dashboards
-

8. Predictive Analytics Subsystem

 **Purpose:** Forecast vegetation, temperature, and urbanization trends.

Components:

-  **Model Trainer:** Trains LSTM/Prophet models on historical NDVI/LST data.
-  **Inference Engine:** Predicts short-term environmental changes.
-  **Integration:** Pushes forecasts to the visualization subsystem.

Interfaces:

- **Input:** Historical data from the Storage subsystem
 - **Output:** `forecast_results` topic + dashboard updates
-

9. Orchestration & Monitoring Subsystem

 **Purpose:** Coordinate, monitor, and maintain system reliability.

Components:

-  **Orchestrator:** Airflow/Prefect DAGs to define and execute workflows.
-  **Metrics:** Prometheus + Grafana for performance and uptime metrics.
-  **Logging:** Centralized logging with ELK Stack or OpenTelemetry.
-  **Health Checks:** Periodic service heartbeat monitoring.

Interfaces:

- **Input:** All subsystems
 - **Output:** Admin dashboards + alert channels
-

Subsystem Summary Table

Subsystem	Key Tools	Core Function
Data Ingestion	Airflow, Kafka, PostGIS	Fetch + queue AOI-specific imagery
Preprocessing	Celery, Redis, Rasterio	Clean + prepare satellite data
Index Computation	Dask, Spark	Calculate NDVI, NDWI, NDBI, LST
Change Detection	Custom Rules Engine	Detect temporal anomalies
Notification	SMTP, Slack API	Alert users of changes
Storage & Caching	TimescaleDB, Redis, S3	Manage imagery + time-series data
Visualization	FastAPI, Streamlit, Leaflet	Interactive dashboards
Predictive Analytics	LSTM, Prophet	Forecast environmental trends
Orchestration & Monitoring	Airflow, Grafana	Manage + monitor pipelines